

DEVOPS PIPELINE WITH DOCKER COMPOSE FOR AUTOMATED INTEGRATION TESTING

A PROJECT REPORT

(22ITP62-Project Work)

Submitted by

JAYASWATI P

23ITR069

in partial fulfilment of the requirements

for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY



PERUNDURAI ERODE – 638 060

MARCH 2026

**DEPARTMENT OF INFORMATION TECHNOLOGY
KONGU ENGINEERING COLLEGE**

(Autonomous)

PERUNDURAI ERODE – 638060

MARCH 2026

BONAFIDE CERTIFICATE

This is to certify that the Project report entitled “**Devops Pipeline With Docker Compose For Automated Integration Testing**” is the Bonafide record of project work done by and **Jayaswathi P (23ITR069)** in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in **INFORMATION TECHNOLOGY** of Anna University Chennai during the year 2025 - 2026.

SUPERVISOR

Date:

HEAD OF THE DEPARTMENT

(Signature with seal)

Submitted for the end semester viva voce examination held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DEPARTMENT OF INFORMATION TECHNOLOGY**KONGU ENGINEERING COLLEGE****(Autonomous)****PERUNDURAI ERODE – 638060****MARCH 2026****DECLARATION**

I affirm that the Project Report titled **“Devops Pipeline With Docker Compose For Automated Integration Testing”** being submitted in partial fulfilment of the requirements for the award of Bachelor of Technology is the original work carried out by me. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

JAYASWATI P
(23ITR069)

Date:

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date:

Name and Signature of the Supervisor with seal

ACKNOWLEDGEMENT

First and foremost, abundant grace and the presence of the Almighty are gratefully acknowledged for guiding the project through its different phases and ensuring its successful completion.

Gratitude is expressed to Correspondent **THIRU. E. R. K. Krishnan, M.Com.**, and all the trust members of Kongu Vellalar Institute of Technology Trust for providing the necessary facilities to complete the project successfully.

A deep sense of gratitude is conveyed to the beloved Principal, **Dr. R. Parameshwaran M.E., Ph.D.**, for providing the opportunity to complete this project.

Sincere thanks are extended to **Dr. S. Anandamurugan M.E., Ph.D.**, Head of the Department, Department of Information Technology, for his valuable suggestions.

I am highly indebted to **Mr. A. P. Ponselvakumar M.E.**, Department of Information Technology, for his valuable supervision and guidance towards the successful completion of this project.

I also express my sincere thanks to all the faculty members of the Department of Information Technology for their valuable guidance and support.

DEVOPS PIPELINE WITH DOCKER COMPOSE FOR AUTOMATED INTEGRATION TESTING

AIM

The aim of this project is to design and implement a **DevOps Pipeline using Docker Compose and Jenkins** to automate the process of **integration testing** by creating a containerized testing environment that ensures faster, reliable, and consistent software testing with minimal manual effort.

ABSTRACT

DevOps is a modern software development methodology that improves collaboration between development and operations teams through automation and continuous integration. In this project, a DevOps pipeline is implemented using **Docker Compose** and **Jenkins** to automate integration testing in a containerized environment.

The objective of this project is to create an automated workflow where applications are built, tested, and validated using Docker containers. Docker Compose helps manage multiple containers required for testing environments, while Jenkins automates the execution of integration tests.

This pipeline ensures faster testing, reduced manual effort, improved reliability, and consistent deployment environments. The implementation demonstrates how container-based integration testing improves software development productivity and reduces errors.

SCOPE AND OBJECTIVES

SCOPE

- Focuses on setting up a real-time system monitoring solution using Prometheus, Node Exporter, and Grafana on an Ubuntu virtual machine.
- Covers the complete process of collecting system metrics, configuring Prometheus, and displaying data through Grafana dashboards.
- Demonstrates how to track critical system metrics such as CPU usage, memory consumption, and disk space in real time.
- Reinforces the practical understanding of how system monitoring tools can be integrated for better visibility and performance tracking.
- Emphasizes data visualization, continuous system health monitoring, and ease of setup using lightweight tools and configurations.

OBJECTIVES

The main objectives of this project are:

- To implement an automated DevOps pipeline
- To configure Docker Compose for test environments
- To integrate Jenkins pipeline automation
- To execute integration testing automatically
- To reduce manual testing effort
- To improve software reliability and testing speed

HARDWARE & SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS

- Processor: Intel i3 or higher
- RAM: Minimum 4GB
- Storage: Minimum 10GB free space

SOFTWARE REQUIREMENTS

- Operating System: Windows / Linux / macOS
- Docker Desktop
- Jenkins
- Git
- Web Browser
- Docker Compose

PROJECT DESIGN

- The project sets up an automated **DevOps pipeline environment** using **Docker Compose and Jenkins** for integration testing.
- Jenkins is configured to automate the build, test, and execution workflow whenever code changes occur.
- Docker Compose is used to create and manage multiple containers required for testing environments.
- Configuration files such as **docker-compose.yml** and **Jenkins pipeline scripts** are written in structured formats for easy automation.
- The pipeline creates isolated container environments for consistent testing across different systems.
- Git repository integration enables automatic triggering of pipeline execution when code is updated.
- The final outcome is a fully automated integration testing pipeline that ensures faster software development and reliable execution.

ADVANTAGES

- Provides automated integration testing workflow.
- Reduces manual effort in environment setup.
- Ensures consistent testing across systems.
- Supports containerized testing environments using Docker Compose.
- Improves development speed and productivity.
- Easy integration with version control systems like Git.
- Detects errors early in development stages.
- Scalable and adaptable for large DevOps environments.

PROPOSED SYSTEM

- Uses **Jenkins pipeline automation** to execute integration testing whenever new code changes are pushed.
- Implements **Docker Compose** to create multi-container testing environments automatically.
- Uses structured configuration files like **docker-compose.yml** for defining services and dependencies.
- Executes integration tests inside containers to maintain environment consistency.
- Automatically starts and stops containers during pipeline execution.
- Supports automated workflows for Continuous Integration (CI).
- Provides scalable architecture suitable for microservices-based applications.

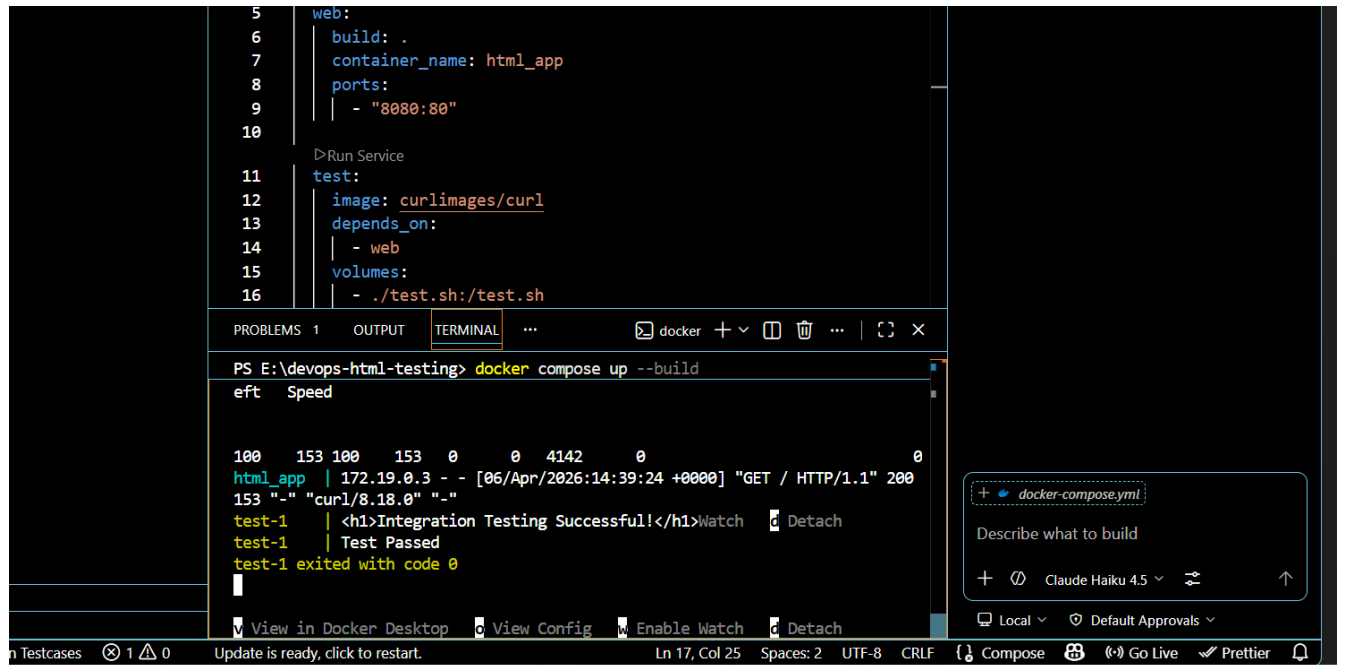
IMPLEMENTATION OF THE PROJECT

The project was implemented by first installing Docker and Jenkins on the system. Jenkins was configured as the automation server to manage the DevOps pipeline workflow. A sample application repository was connected to Jenkins for automated testing. Docker Compose was then configured using a YAML file to create multiple containers required for integration testing environments.

The Jenkins pipeline script was created to automate stages such as cloning the repository, building containers, running integration tests, and stopping containers after execution. Whenever changes were pushed to the repository, Jenkins automatically triggered the pipeline execution. Docker Compose created isolated testing environments and executed the required tests successfully.

The entire setup ensured consistent testing environments and reduced manual configuration effort. The result was a lightweight, automated, and reliable DevOps pipeline capable of executing integration tests efficiently.

OUTPUT SCREENSHOTS



The screenshot shows a VS Code editor with a Docker Compose configuration file and its terminal output. The configuration file defines a 'web' service and a 'test' service. The terminal shows the command 'docker compose up --build' being executed, followed by the output of the 'test' service, which includes a successful integration test result.

```
5 web:
6   build: .
7   container_name: html_app
8   ports:
9     - "8080:80"
10
11 > Run Service
12 test:
13   image: curlimages/curl
14   depends_on:
15     - web
16   volumes:
17     - ./test.sh:/test.sh
```

```
PS E:\devops-html-testing> docker compose up --build
eft Speed

100 153 100 153 0 0 4142 0
html_app | 172.19.0.3 - - [06/Apr/2026:14:39:24 +0000] "GET / HTTP/1.1" 200
153 "-" "curl/8.18.0" "-"
test-1 | <h1>Integration Testing Successful!</h1>Watch
test-1 | Test Passed
test-1 exited with code 0
```

Fig: DOCKER COMPOSE

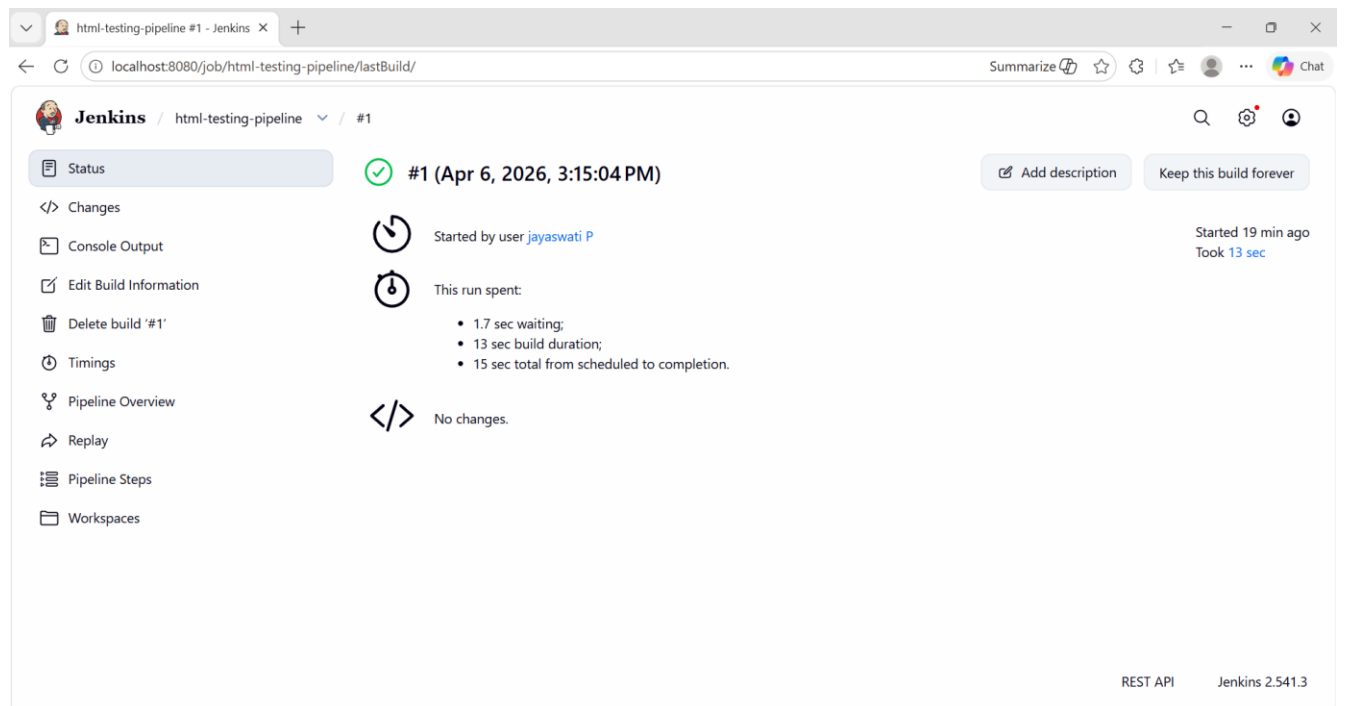


Fig: JENKINS

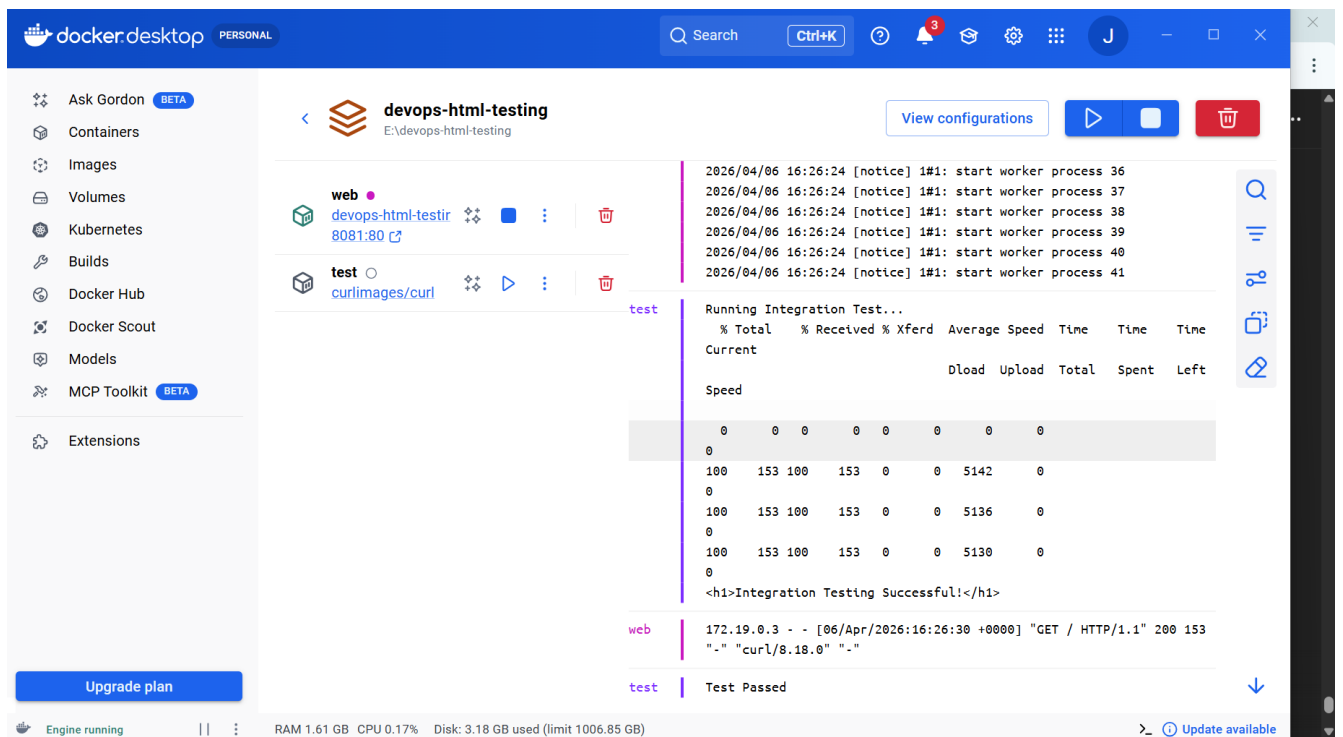


Fig: DOCKER DESKTOP

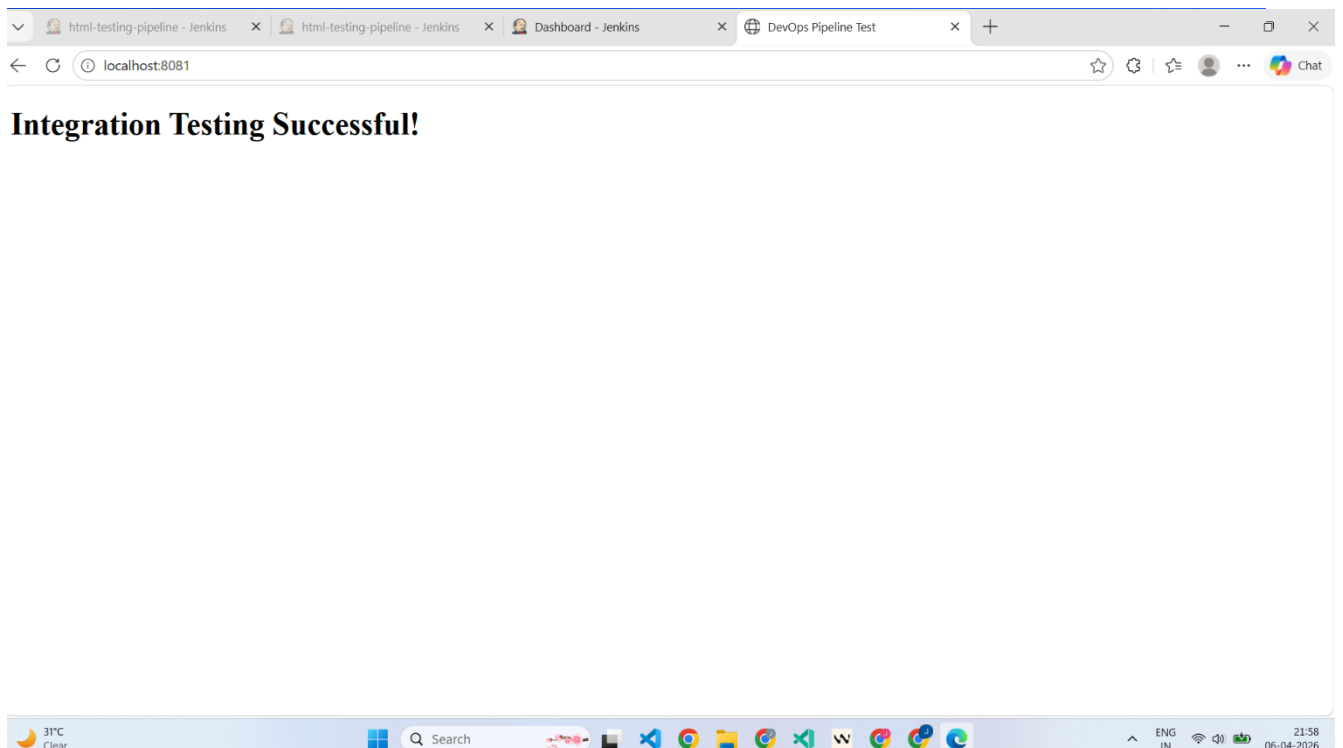


Fig: FINAL OUTPUT

CONCLUSION

Implementing a DevOps pipeline using Jenkins and Docker Compose provides an efficient solution for automated integration testing. Jenkins automates the pipeline workflow, while Docker Compose creates consistent containerized environments for executing tests.

This approach ensures faster development cycles, reduces manual configuration effort, and improves testing reliability. The pipeline supports scalable testing environments and can be extended for Continuous Deployment in future applications. Overall, the project demonstrates how container-based DevOps automation enhances software quality and productivity.

COMMANDS

1. Install Docker

```
sudo apt update  
sudo apt install docker.io
```

Start Docker:

```
sudo systemctl start docker  
sudo systemctl enable docker
```

2. Install Docker Compose

```
sudo apt install docker-compose
```

Verify installation:

```
docker-compose --version
```

3. Run Jenkins Container

```
docker run -p 8080:8080 jenkins/jenkins:lts
```

Access Jenkins:

<http://localhost:8080>

4. Create docker-compose.yml File

```
version: '3'
```

```
services:
```

```
  web:
```

```
    image: nginx
```

```
    ports:
```

```
      - "8081:80"
```

```
  test:
```

```
    image: alpine
```

```
    command: echo "Integration Testing Successful"
```

Run Docker Compose:

```
docker-compose up -d
```

5. Jenkins Pipeline Script

```
pipeline {
```

```
  agent any
```

```
  stages {
```

```
    stage('Clone Repository') {
```

```
      steps {
```

```
        git 'https://github.com/sample/project.git'
```

```
      }
```

```
    }
```

```
    stage('Run Docker Compose') {
```

```
      steps {
```

```
        sh 'docker-compose up -d'
```

```
      }
```

```
    }
```

```
    stage('Run Tests') {
```

```
      steps {
```

```
        sh 'docker-compose run test'
```

```
      }
```

```
    }
```

```
    stage('Stop Containers') {
```

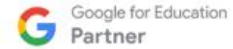
```
      steps {
```

```
        sh 'docker-compose down'
```

```
      }
```

```
    }
```

MASTERING DEVOPS COURSE COMPLETION CERTIFICATE



Certificate of Completion

JAYASWATI P

is here by awarded the certificate of achievement
for the successful completion of

Mastering DevOps

A handwritten signature in blue ink, reading "M. Arunprakash".

M. Arunprakash

Founder and CEO,
GUVI Geek Networks.

Certificate issued on : **March 19 2026**

Valid certificate ID : **791rn3h9ZFLS8191X7**

Verify certificate at: www.guvi.in/certificate?id=791rn3h9ZFLS8191X7

REFERENCES

- Docker Official Documentation
<https://docs.docker.com>
- Jenkins Official Documentation
<https://www.jenkins.io/doc/>
- Docker Compose Documentation
<https://docs.docker.com/compose/>
- DevOps Concepts Guide
<https://azure.microsoft.com/en-in/resources/devops-guide>
- GitHub Repository Link - <https://github.com/jayaswatip/devops-html-testing>